

Universidade de Caxias do Sul (UCS)
Área de Ciências Exatas e Tecnologia
Disciplina: Programação de Computadores I

Linguagem de Programação C

Comando Condicional

Leonardo Pellizzoni

Comando Condicional

- Até o momento, construímos programas que executam todos os comandos do programa de forma sequencial
- Normalmente o que ocorre é que, dependendo dos dados recebidos, o programa deve executar comandos diferentes, resultando em "comportamentos" diferentes.

Comando Condicional

- Exemplo - cálculo das raízes de uma equação de segundo grau:
 - Se os valores de a , b e c são tais que a equação não tem raízes reais, $b^2 - 4ac$ resultará em um valor negativo, e a extração de sua raiz quadrada ocasionará um erro de execução.
 - Um programa mais robusto deveria testar o sinal de $b^2 - 4ac$ e calcular as raízes somente se fosse maior ou igual a zero.

Comando Condicional

- Para que um programa possa executar comandos diferentes em diferentes situações usa-se o comando *if*. Seu formato é:

if (condição)

{

comandos;

}

- Esse trecho de programa pode ser lido como "Se a condição é verdadeira, então execute os comandos"

Comando Condicional

- **condição** é uma expressão de comparação que resulta verdadeiro ou falso e utiliza, para comparação, os **operadores relacionais** (operadores que identificam a relação entre dois valores):
 - **igual** : representado por `=`
 - **diferente** : representado por `!=`
 - **maior, menor, maior ou igual, menor ou igual**: `>`, `<`, `>=`, `<=`

Comando Condicional

- O objetivo das chaves na estrutura

if (condição)

{

comandos;

}

- é agrupar os comandos dentro das chaves em um único bloco, de modo que todos estejam "dentro" do comando *if*.
- Se houver apenas um comando a ser executado, as chaves podem ser dispensadas usando o formato a seguir:

if (condição)

comando;

Comando Condicional

- Exemplos:

```
if (a >= 0)
```

```
    printf("Positivo\n");
```

```
if (a > b)
```

```
    printf("O maior e %dn\n",a);
```

```
if (a > b)
```

```
{
```

```
    printf("O maior e %dnn",a);
```

```
}
```

- Os dois últimos são equivalentes, para mostrar que as chaves são opcionais quando há apenas um comando.

Comando Condicional

- **ERRO!!!** Se houver mais de um comando "dentro" do if e não houver chaves, apenas o comando imediatamente após o if estará dentro do if. Todos os comandos a partir do segundo comando estarão fora do if e serão executados sempre, independentemente da condição ser verdadeira ou falsa.

- Exemplo:

if (a >= 0)

printf("Positivo\n");

printf("Esse printf está fora do if");

Comando Condicional

- **ERRO!!!** Um outro erro muito comum é colocar um ponto-e-vírgula após a condição como no exemplo:

```
if (a >= 0);
```

```
printf("Positivo\n");
```

- Nesse exemplo, o ponto-e-vírgula termina o comando if, de modo que o printf está fora do if e será executado sempre, independentemente da condição ser verdadeira ou falsa

Comando Condicional

- Quando um conjunto de comandos deve ser executado se a condição for verdadeira e um conjunto diferente deve ser executado se a condição for falsa, utiliza-se o formato:

if (condição)

{

lista 1 de comandos;

}

else

{

lista 2 de comandos;

}

- Esta forma pode ser lida como "Se a condição é verdadeira, então execute a lista 1 de comandos, senão (se a condição é falsa), execute a lista 2 de comandos"

Comando Condicional

- Exemplo:

```
if (a >= 0)
```

```
    printf("Positivo\n");
```

```
else
```

```
    printf("Negativo\n");
```

```
if (a > b)
```

```
    printf("O maior e %d\n",a);
```

```
else
```

```
    printf("O maior e %d\n",b);
```

Comando Condicional

- Se a lista de comandos do if ou do else tiver mais de um comando, eles devem ser agrupados utilizando chaves
- Se a lista do if ou do else tiver apenas um comando, as chaves são desnecessárias (mas podem ser utilizadas mesmo com um único comando):

```
if (a > 0)
```

```
{
```

```
    printf("Sou positivo");
```

```
    b=0;
```

```
}
```


Comando Condicional

- O comando condicional deve ser utilizado quando há diversas situações que devem ser tratadas de formas diferentes pelo programa. Os passos para utilizá-lo são:
 1. Identificar com clareza quais as diferentes situações que devem ser tratadas pelo programa
 2. Definir qual o comportamento (comandos que devem ser executados) que o programa deve ter em cada uma das situações
 3. Encontrar expressões que permitam ao programa identificar cada uma das situações

Operadores Lógicos

- Há situações em que uma única comparação não é suficiente para o programa identificar tratar a situação
- Uma situação simples seria a de encontrar, através de comparações, o maior valor entre 3 valores **a**, **b** e **c**
- Para saber se o valor na variável **a** é o maior de todos, é necessário compará-lo com os valores em **b** e **c**.
 - “Se **a** é maior que **b** E **a** é maior que **c**...”
- Ou pode-se querer saber, se entre 3 valores, algum é igual a zero:
 - “Se **a**==0 OU **b**==0 OU **c**==0...”

Operadores Lógicos

- Nessas situações, para poder colocar mais de uma comparação, utiliza-se os chamados operadores lógicos, que combinam duas ou mais comparações para gerar um único resultado verdadeiro ou falso.
- Os operadores lógicos em C são 3:
 - O operador **E** (representado por **&&**). Quando aplicado a duas comparações, resulta verdadeiro se e somente se ambas as comparações são verdadeiras
 - O operador **OU** (representado por **||**). Quando aplicado a duas comparações, resulta verdadeiro se pelo menos uma delas é verdadeira (possivelmente as duas)
 - O operador de **negação** (representado pelo símbolo **!**) inverte o valor-verdade da expressão

Operadores Lógicos

- Uma condição pode conter diversas expressões de comparação combinadas com o uso desses 3 operadores
- Ex: *if (a>0 && b>0)...* *if (a>0 || b>0)...*
- O operador **&&** tem **prioridade maior** que o operador **||**
- Se for necessário comparar um valor com outros dois (ex: $a > b > c$) deve-se efetuar as duas comparações separadamente e combiná-las com **&&** ($a > b$ && $b > c$)

Exemplo 1

- Desenvolva um programa que solicite ao usuário digitar um número inteiro. Após a digitação, o computador deverá mostrar na tela uma mensagem informando se o número é positivo ou negativo
- Nesse caso um único "if" é suficiente para diferenciar as duas situações:

if ($a > 0$)

printf("Positivo");

else

printf("Negativo");

Exemplo 1

- Pode ocorrer também que no caso de a condição falhar, nada deve ser feito.
 - Nesse caso o "else" pode ser omitido
if (a > b)
printf("%d e maior", A);

Exemplo 2

- Desenvolva um programa que leia um valor e escreva:
 - 0, se o valor é zero
 - 1, se o valor é maior que zero
 - -1 - se o valor é negativo
- Nesse caso um único "if" não é suficiente para diferenciar as diferentes situações
- Pode-se utilizar um "if" para identificar cada situação
 - Nesse caso, a expressão deve ser suficiente para identificar cada caso dentre todos os casos

Exemplo 2

- Nesse exercício podem ocorrer 3 situações distintas, que exigem tratamentos distintos:
 - O valor lido é maior que 0
 - O valor lido é igual a 0
 - O valor lido é menor que 0

Exemplo 2

- Pode ser resolvido com 3 comandos if, um para cada situação:

```
printf("Digite um valor:");
```

```
scanf("%d",&a);
```

```
if (a>0)
```

```
    printf("1");
```

```
if (a==0)
```

```
    printf("0");
```

```
if (a<0)
```

```
    printf("-1");
```